A Likelihood-Based Pretraining and Refinement Algorithm for Probit Independent-Mixture Factor Models

September 1, 2026

1 Why Estimate the Signal Before Augmenting?

The key advantage of the likelihood-based pretraining step is that it targets the identifiable signal first. In the probit factor model,

$$\mathbb{P}(X_{ij} = 1) = \Phi(\alpha_j + B_{ij}), \qquad B = F\Lambda^\top,$$

the systematic low-rank object is B. By contrast, an augmented latent Gaussian draw satisfies

$$Z_{ij} = \alpha_j + B_{ij} + \epsilon_{ij}.$$

Thus an SVD of an augmented Z matrix is trying to recover the low-rank signal from a noisy latent realization. Even if the augmentation is correctly specified, the sampled noise can perturb the singular vectors, introduce Monte Carlo variation, and slow the initialization.

The alternative is to estimate B directly from the binary probit likelihood and then take the SVD of $\widehat{B}$. This gives a cleaner initial estimate of the signal subspace, avoids repeated stochastic augmentation during pretraining, and provides a more direct theoretical route: if $\widehat{B}$ consistently estimates the low-rank response surface, then its singular subspace consistently estimates the latent factor subspace. The subsequent rotation and refinement stages then operate on a likelihood-based estimate of the systematic signal rather than on noisy augmented draws.

2 Goal

The main goal of pretraining is not to estimate uniquely identified factors and loadings. It is to estimate the low-rank probit response surface. We write

$$\Theta = \mathbf{1}_n \alpha^\top + B, \qquad B = F\Lambda^\top, \qquad \operatorname{rank}(B) \le H.$$

The product B is identified by the binary likelihood, while a particular factorization $B = F\Lambda^\top$ is not unique. This distinction is the reason for the staged strategy: first estimate B and its subspace, then rotate that subspace into mixture-factor coordinates, and finally refine the full structured model.

3 Stage 1: Estimate the Low-Rank Signal

For a supplied or selected working rank H, the first stage estimates (α, B) by maximizing the rank-constrained probit likelihood

$$(\widehat{\alpha}, \widehat{B}) \in \underset{\alpha,\, B:\ \operatorname{rank}(B) \le H,\ \mathbf{1}_n^\top B = 0}{\arg\max} \ell_H(\alpha, B), \tag{1}$$

where

$$\ell_H(\alpha, B) = \sum_{i=1}^{n} \sum_{j=1}^{p} \left[X_{ij} \log \Phi(\alpha_j + B_{ij}) + (1 - X_{ij}) \log\{1 - \Phi(\alpha_j + B_{ij})\} \right]. \tag{2}$$

The centering constraint separates item intercepts from the low-rank interaction. Under mild regularity conditions, this likelihood-based step recovers the population rank-H signal subspace. Taking the SVD of $\widehat{B}$ then gives a consistent initialization of the left singular subspace; when the working rank matches the data-generating rank, this is the true factor-score subspace.

Computationally, this can be done with a simple deterministic EM-SVD algorithm. Given the current low-rank signal, the E-step computes

$$W_{ij} = \mathbb{E}\{Z_{ij} \mid X_{ij}, \alpha_j + B_{ij}\},$$

using the usual truncated-normal probit formula. The M-step projects W back onto the constrained low-rank class: set $\widehat{\alpha} = \text{colMeans}(W)$, center W by columns, and take the rank-H SVD. Thus the rank constraint is enforced by SVD truncation and the intercept constraint is enforced by column-centering.

This removes the need to use stochastic augmented-Z draws as the pretraining estimator. The latent-Gaussian representation is still useful for EM calculations, but the target is the observed binary likelihood in (1). In practice this is faster because pretraining no longer repeatedly samples full $n \times p$ augmented matrices and recomputes the rotation from noisy draws. Instead, each EM iteration uses deterministic conditional means followed by a low-rank SVD projection.

4 Stage 2: Rotate the Subspace

After estimating $\widehat{B}$, we take a convenient factorization, for example from the SVD,

$$\widehat{B} = \widehat{F}\widehat{\Lambda}^{\top}.$$

These scores estimate the correct subspace, but their axes are arbitrary up to rotation. The mixture assumption supplies the missing orientation: we look for the rotation under which the score coordinates look like independent one-dimensional mixtures. This is the ICA-like step of the algorithm.

5 Stage 3: Refine the Structured Model

Starting from the rotated scores, the refinement stage updates the full product-mixture factor model:

$$\alpha, \quad F, \quad \Lambda, \quad \{\pi_{hg}, \mu_{hg}, \sigma^2_{hg}\}.$$

In our current implementation, refinement uses MAP mixture updates and can include sparsity penalties on the loadings. The practical point is that refinement starts from a likelihood-based estimate of the rank-H subspace, rather than from a noisy sampled augmentation. The refinement step is where the model moves from a good low-rank probit representation to the full independent-mixture factor model used for interpretation and prediction.

Reporting convention. The final fitted model is still invariant to location, scale, sign, and column ordering conventions. After convergence, we apply a canonical normalization only for reporting. If m_h and s_h are the fitted marginal mixture mean and standard deviation for factor h, set

$$f^\star_{ih} = \frac{f_{ih} - m_h}{s_h}, \qquad \lambda^\star_{jh} = s_h \lambda_{jh}, \qquad \alpha^\star_j = \alpha_j + \sum_h \lambda_{jh} m_h.$$

This preserves the probit linear predictor:

$$\alpha_j + \boldsymbol{f}_i^\top \boldsymbol{\lambda}_j = \alpha^\star_j + (\boldsymbol{f}^\star_i)^\top \boldsymbol{\lambda}^\star_j.$$

So the fitted probabilities do not change; only the parameter convention changes.

6 Code Pointers

The main implementation is `R/probit_ifa_em_svd_pretraining.R`, which contains the low-rank EM-SVD fit, the mixture-rotation pretrainer, and the pretrain-then-refine wrapper. The simulation checks are in `scripts/sample_size/`: `test_em_svd_pretraining_simulation.R` and `test_final_canonical_normalization.R`.